A Robust Vision-based Algorithm for Classifying and Tracking Small Orbital Debris Using On-board Optical Cameras

Yasin Zamani
University of Utah, Salt Lake City, UT
Joel Amert
NASA/MSFC, Huntsville, AL

Thomas Bryan
NASA/MSFC, Huntsville, AL

Neda Nategh
University of Utah, Salt Lake City, UT

ABSTRACT

This study develops a vision-based classification and tracking algorithm to address the challenges of in-situ small orbital debris environment tracking, including debris observability and instrument requirements for small debris observation. The algorithm operates in near real-time and is robust under challenging tasks in moving objects tracking such as multiple moving objects, objects with various movement trajectories and speeds, very small or faint objects, and substantial background motion. The performance of the algorithm is optimized and validated using space image data available through simulated environments generated using NASA Marshall Space Flight Center's Dynamic Star Field Simulator of on-board optical sensors and cameras.

1. INTRODUCTION

After more than 50 years of human space activities, orbital debris has become a serious problem in the near-Earth environment [1]. The U.S. Space Surveillance Network is currently tracking more than 22,000 objects larger than 10 centimeters (cm). Additional optical and radar data indicate that there are about 500,000 pieces of debris larger than 1 cm and more than 100 million pieces of debris larger than 1 millimeter (mm) in orbit. Currently, space debris is mainly tracked using on-Earth sensors including radar systems and optical telescopes. The National Aeronautics and Space Administration (NASA) and Department of Defense (DoD) cooperatively characterize the orbital environment. DoD's Space Surveillance Network actively tracks approximately 20,000 pieces of debris orbiting Earth with a size greater than 10 cm. NASA uses additional ground-based sensors and inspections of returned satellite surfaces to estimate the number of untracked debris particles smaller than 10 cm. NASA's statistical analysis predicts there are between 300,000 to 600,000 debris particles orbiting earth that are smaller than a softball. These small, untracked debris objects ($\sim$500,000) outnumber the larger and tracked objects ($\sim$20,000) by a ratio 25 to 1. They are too small to track but too large to shield against. Therefore, the risk of the small untracked debris objects to operational spacecraft is much higher than the risk posed by the larger and tracked debris objects. Low Earth Orbit (LEO) has the highest concentration of both active satellites and debris, thus increasing the probability of objects colliding. Figure 1 shows the estimated locations of space debris, highlighting their concentration in LEO.

The 2015 NASA Technology Roadmaps identify the critical gap in required technologies for orbital debris tracking and characterization. The requirements identified are technologies to: (1) track and characterize debris that is 10 to 100 times smaller than debris that is currently being tracked; and (2) reduce tracking time to accommodate the larger number of targets being tracked.

NASA Marshall Space Flight Center (MSFC) has focused its efforts on small orbital debris ranging in size from 5 mm to 10 cm as it was shown that debris of these sizes present the greatest threat to operational spacecraft as they are too

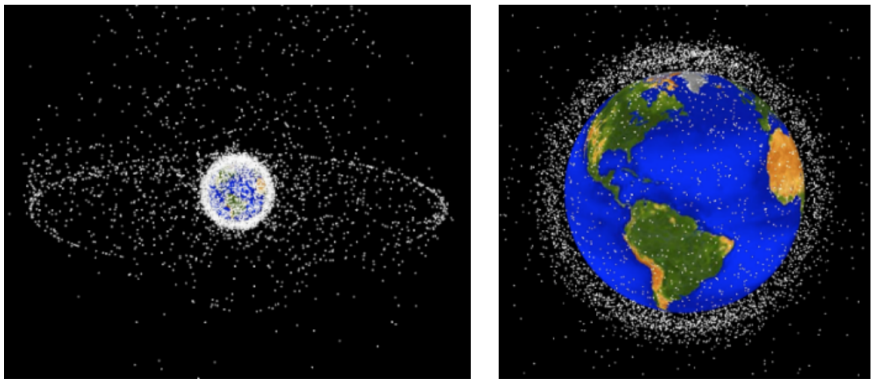


Fig. 1: Computer generated orbital debris graphics displaying currently tracked debris objects. Approximately 95% of the objects in this illustration are orbital debris, i.e., not functional satellites. The dots represent the current location of each item. (Left) Geosynchronous Earth Orbit (GEO) (Right) Low Earth Orbit (LEO).

small to track but too large to shield against. Analysts at MSFC have begun to characterize the small debris environment in LEO to support several spacecraft concept studies and hardware test programs addressing the characterization, mitigation, and ultimate removal of small debris. The Small Orbital Debris Active Removal (SODAR) architectural study investigated the overall effectiveness of removing small orbital debris from LEO using a low power, space-based laser.

The Small Orbital Debris Detection, Acquisition, and Tracking (SODDAT) conceptual technology demonstration spacecraft was developed to address the challenges of in-situ small orbital debris environment classification including debris observability and instrument requirements for small debris observation. These studies have identified the ability to detect and precisely track small LEO object from a space-based platform as a critical technology requirement, which is also highlighted in this proposal solicitation as one of Marshall Technical Requirements supporting spacecraft systems.

To address this challenge, this study, defined in conjunction with scientists at MSFC, focuses on the problem of detection and tracking for small orbital debris ranging in size from 5 mm to 10 cm using a new low-cost space-based orbital debris tracking system. The algorithm technology development builds on the SODDAT conceptual technology demonstration developed by MSFC.

The vision-based system developed here is composed of two major components, a classification algorithm and a tracking algorithm. The classification algorithm takes the pixel location and brightness information of everything in the field of view (FOV) of an optical camera captured in a video sequence. This pixel information is used to compute the centroid location of each spot in each camera frame and their associated brightness values. The centroid data are then used by a statistical classifier for differentiating between stars and debris objects in the set of detected spots. The classifier takes successive camera frames as its input and determines which detected values are debris objects. Next, the tracking algorithm locates classified debris objects in consecutive frames and track them over time. The algorithms should perform accurately and robustly in challenging environments such as moving star background, multiple moving objects, objects with various moving trajectories and speeds, occlusion of spots. The algorithm should also be computationally tractable for on-orbit calculations. The accuracy of the algorithm is evaluated across a variety of image conditions and movement patterns and dynamics.

Moreover, we optimize and validate the performance of the algorithm using testing capabilities and space debris images provided by the MSFC. The MSFC's one-of-a-kind Dynamic Star Field Simulator (DSFS) uses a high resolution large monochrome display and a custom collimator capable of projecting realistic star images with simple orbital debris spots (down to star magnitude 11-12) into a passive orbital detection and tracking system with simulated real-time angular motions of the vehicle-mounted sensor. The DSFS can be expanded for multiple sensors (including advanced star trackers), real-time vehicle pointing inputs, and more complex orbital debris images. Images from the DSFS serve as inputs to the detection algorithm to track simulated small orbital debris objects.

This vision-based classification and tracking algorithm will be used to detect and track very small and faint space debris, very quickly and robustly, using videos captured by an optical camera from an orbit. The ultimate tracking system can be used on a constellation of small satellites in LEO, which will enable detecting debris outside of the presence of atmospheric scintillation to achieve higher precision than existing systems [2, 3].

2. METHODS

2.1 Classification Algorithm

The purpose of the proposed classification algorithm is to identify the bright spots in captured video sequences and to classify them as "objects" (representing orbital debris objects) or "stars". In cases the algorithm is not able to classify any of the detected spots into one of the objects or stars classes, that spot will be labeled as "unknowns", which is neither the objects class nor the stars class. The input to the algorithm is a sequence of intensity images over time, which represents the captured space videos. Algorithm 1 and Fig. 2 describe the classification algorithm's components.

Algorithm 1 Classification

function CLASSIFY(V)
  Input: Video V ▷ Sequence of intensity images
  Output: Labels L, centroids C ▷ Each label could be *'Object'*, *'Star'*, or *'Unknown'*
  constant τ ▷ Threshold of binarization
  constant c ▷ Cost of *Unknown* label (not matching)
  constant d ▷ Delay between two consecutive frames
  constant α ▷ Significance level for detecting objects
  constant K ▷ Maximum number of objects

  for $i = 1$ **to** d **do** ▷ For the first d frames
    $C_i \leftarrow$ CENTROIDS(V_i, τ)
    $T_i \leftarrow \{[0,0]\}$ ▷ Translation vectors: $[t_x, t_y]$
    $R_i \leftarrow \{[0,0,0]\}$ ▷ Rotations: $[c_x, c_y, \theta]$
    $L_i \leftarrow \emptyset$
  end for

  for $i = d+1$ **to** $|V|$ **do** ▷ For the frame $d+1$ to the end
    $C_i \leftarrow$ CENTROIDS(V_i, τ)
    $D \leftarrow$ DISTANCEMATRIX($C_{i-d}, C_i, T_{i-d}, R_{i-d}$)
    $M_i \leftarrow$ MATCHES(D, c)
    $T_i \leftarrow$ TRANSLATIONS(C_{i-d}, C_i, M_i)
    $R_i \leftarrow$ ROTATIONS($C_{i-2d}, C_{i-d}, C_i, M_{i-d}, M_i$)
    $[\theta, \rho, \delta] \leftarrow$ FEATURES(T_i, R_i)
    $O \leftarrow$ OUTLIERS(θ, α, K) $\bigcup$ OUTLIERS(ρ, α, K) $\bigcup$ OUTLIERS(δ, α, K)
    $L_i \leftarrow$ LABELS(C_i, M_i, O)
  end for
  return L, C
end function

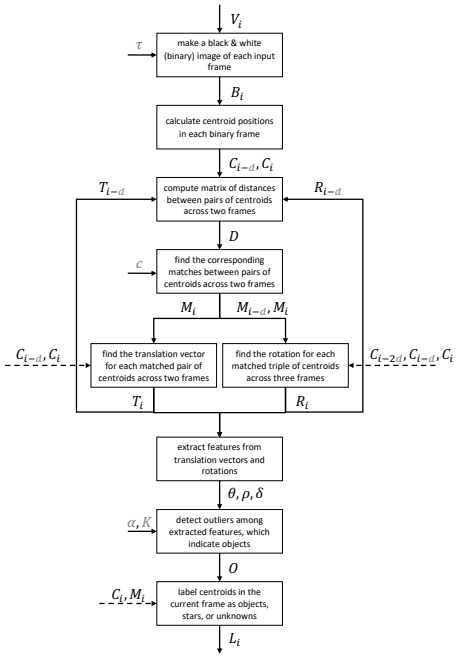
To classify spots in each image frame, the first step is to determine the position of the spots in each frame as follows. Using grayscale images as the input, each pixel value between 0 and 1 in the image represents the light intensity at that pixel, with larger values representing brighter pixels. A global intensity threshold value is chosen such that it can remove as many noisy pixels as possible in all the input frames. Using this threshold value, input grayscale images are converted into binary black and white images [4].

To determine the centroid of each spot using the detected white pixels, we find clusters of white pixels that are one of the 8-connectivity neighborhoods of each other. Each cluster representing a set of connected components specify an area in the image that each spot covers. Each spot is then identified by the center of its corresponding

Block Diagram Notations:

- $V_i \rightarrow i^{th}$ intensity/gray-scale video frame (in the block diagram, V_i refers to the current frame, and V_{i-d} refers to the dth frame before the current frame).
- $\tau \rightarrow$ Threshold value selected to make a black and white (binary) image from the intensity image.
- $B_i \rightarrow i^{th}$ black and white (binary) frame.
- $C_i \rightarrow$ Spot centroids for the ith binary frame.
- $D \rightarrow$ Distance matrix. $D_{p,q} \rightarrow$ Euclidean distance between the pth spot centroid in the previous frame with q^{th} spot centroid of current frame.
- $c \rightarrow$ Cost value selected for labeling a spot as *'Unknown'* (no matching found).
- $M_i \rightarrow$ Matches contains (p,q) pairs such that p^{th} spot centroid in the previous frame is matched with the qth spot centroid in the current frame.
- $T_i \rightarrow$ Translation vectors calculated for the ith frame. For each matched pair, there is a translation vector defined from the centroid in the previous frame to the matched centroid in the current frame.
- $R_i \rightarrow$ Rotations calculated for the ith frame. For each matched triple in three consecutive frames, there is a rotation defined from the centroid in the previous frame to the matched centroid in the current frame.
- $[\theta, \rho, \delta] \rightarrow$ Translation directions θ, translation magnitudes ρ, and distance from median of center of rotations δ are extracted classifcation features.
- $O \rightarrow$ Outliers is a logical array whose elements are true when an outlier is detected in the corresponding extracted features.
- $\alpha \rightarrow$ Significance level is the probability of the study rejecting the null hypothesis, given that the null hypothesis were true.
- $K \rightarrow$ Maximum outlier count specifies the maximum number (upper bound) of outliers returned by the method.
- $L_i \rightarrow$ Label of the spots in the ith frame. Each label could be *'Object'*, *'Star'*, or *'Unknown'*.

(a)



(b)

Fig. 2: Block diagram of classification algorithm.

connected components, called centroids [5]. The centroids information is calculated for each frame (Algorithm 2). The classification algorithm uses the centroids information at a given frame (referred to as current frame in the text) as well as that of in the d-th frame before (referred to as previous frame), where d represents the history of the centroids information that the algorithm uses for its computations.

Algorithm 2 Spot Centroids

function CENTROIDS(I, τ)
Input: Intensity image I, binarization threshold τ
Output: Centroids C ▷ Each centroid is an (x,y) pixel position
    $B \leftarrow$ Binarize intensity image I by using threshold τ
    for all connected component $W \in B$ **do**
        append (x,y) coordinate of the centroid for the connected component W to C
    end for
    return C
end function

Next, the distance of each center location of spots in the current frame to those in the previous frame is calculated (Algorithm 3). It should be noted that this algorithm uses the history of motion of the spots to calculate the distance matrix since this information helps to find more accurate correspondences in the next step. This analysis generates a distance matrix whose entries evaluate the Euclidean distance between one centroid in the current frame to another one in the previous frame [6]. The distance values represent the cost associated with matching two centroids between the current and previous frame.

Algorithm 3 Distance Matrix

function DISTANCEMATRIX(C_1, C_2, T, R)
Input: Centroids of previous C_1 and current frame C_2, expected translation vector T and rotation R
Output: Distance matrix D
  if R is a valid rotation **then** ▷ R is a valid rotation if $c_x, c_y \neq \infty \wedge \alpha \neq 0$
    $C_1 \leftarrow$ apply R on C_1
  else
    $C_1 \leftarrow$ apply T on C_1
  end if
  for $i = 1$ **to** $|C_1|$ **do**
    for $j = 1$ **to** $|C_2|$ **do**
      $D_{i,j} \leftarrow \left\| C_{1,i} - C_{2,j} \right\|_2$
    end for
  end for
  return D
end function

Using the distance matrix, we obtain the correspondence between centroids in the current frame versus those in the previous frame as described in Algorithm 4.

It should be noted that these correspondences must be one-to-one, i.e., each centroid in the current frame corresponds to one and only one centroid at maximum in the previous frame; equivalently, each centroid in the previous frame should also have a maximum of one matched centroid in the current frame [7, 8]. Those centroids in the current frame for which a match cannot be found, for example in the case that a spot has just entered the image frame and was not present in the previous frame, are labeled as unknowns in that frame.

Algorithm 4 Match Pairs
Generalized Linear Assignment Problem

function MATCHES(D, c)
Input: Distance matrix D, unmatched cost c
Output: Match pairs M

$$\text{minimize} \sum_{i=1}^{m}\sum_{j=1}^{n} D_{ij}x_{ij} + c \cdot (m+n-2|M|)\,.$$

$$\text{subject to} \sum_{j=1}^{n} x_{ij} \leq 1 \qquad i = 1,\ldots,m;$$

$$\sum_{i=1}^{m} x_{ij} \leq 1 \qquad j = 1,\ldots,n;$$

$$x_{ij} \in \{0,1\} \qquad i = 1,\ldots,m, \quad j = 1,\ldots,n;$$

$$M = \left\{(i,j)\,|\,x_{i,j} = 1\right\} \qquad i = 1,\ldots,m, \quad j = 1,\ldots,n.$$

  return M
end function

Using the matched pairs, we calculate a translation vector for each centroid in the current frame relative to its corresponding matched centroid in the previous frame (Algorithm 5). Each translation vector represents a shift in the location of a centroid from the previous frame to the current frame [9].

Algorithm 5 Translation Vectors

function TRANSLATIONS(C_1, C_2, M)
Input: Centroids of previous C_1 and next frame C_2, and match pairs M
Output: Translations T
  for all $(i, j) \in M$ **do**
    $[t_x, t_y] \leftarrow C_{2,j} - C_{1,i}$
    append $[t_x, t_y]$ to T
  end for
  return T
end function

For each triplet of matched centroids in three consecutive frames, we define a rotation using a circle construction through three noncollinear points, also known as the circumscribed circle of a triangle defined by those three matched points (Algorithm 6). This rotation determines the trajectory along which a centroid rotates from the previous frame to the current frame [10].

Algorithm 6 Rotations

function ROTATIONS(C_1, C_2, C_3, M_1, M_2)
Input: Centroids of three consecutive frames C_1, C_2, $C3$, and match pairs M_1 and M_2
Output: Rotations R
  for all $(i, j) \in M_1$ **and** $(j, k) \in M_2$ **do**
    $[c_x, c_y, \alpha] \leftarrow$CENTERANGLEOFROTATION($C_{1,i}, C_{2,j}, C_{3,k}$)
    append $[c_x, c_y, \alpha]$ to R
  end for
  return R
end function

function CENTERANGLEOFROTATION(P_1, P_2, P_3)
Input: Two-dimensional points P_1, P_2, and P_3
Output: Center $[c_x, c_y]$ and angle α of rotation
  $L_1 \leftarrow$ Perpendicular bisector of two points P_1 and P_2
  $L_2 \leftarrow$ Perpendicular bisector of two points P_2 and P_3
  $[c_x, c_y] \leftarrow$ Point of intersection of two lines L_1 and L_2
  $\alpha \leftarrow$ Angle between two vectors $P_2 - [c_x, c_y]$ and $P_2 - [c_x, c_y]$
  return $[c_x, c_y, \alpha]$
end function

Algorithm 7 extracts three classification features, including the direction and magnitude of translation vectors as well as the distance from the median of the center of rotations.

Algorithm 7 Feature Extraction

function FEATURES(T, R)
Input: Translation vectors T, rotations R
Output: Translation directions θ, translation magnitudes ρ, and distance from median of center of rotations δ
  $\theta \leftarrow \{\arctan(u) \,|\, u \in T\}$
  $\rho \leftarrow \{\|u\|_2 \,|\, u \in T\}$
  $\delta \leftarrow \{\|[c_x, c_y] - \mu\|_2 \mid [c_x, c_y] \in R, \mu = \text{median}(R)\}$
  return $[\theta, \rho, \delta]$
end function

Using a hypothesis testing approach [11–13], spot centroids with statistically different translation vectors in magnitude or direction and a different distance from the median of the center of rotations are classified into the objects class and

labeled as an object, and the remainder of the spots are classified and labeled as stars. The statistical test and the labeling procedure are described in Algorithm 8 and 9, respectively.

Algorithm 8 Outliers
Generalized (extreme Studentized deviate) ESD test

function OUTLIERS(X)
Input: Sample data X
Output: Index of outliers O, significance level α, maximum number of outliers K
  $XX \leftarrow X$ ▷ Save a copy of X
  for $i = 1$ **to** K **do**
    $\bar{X} \leftarrow \text{mean}(X)$
    $s \leftarrow \text{std}(X)$
    $x^* \leftarrow \underset{x \in X}{\text{argmax}} |x - \bar{X}|$
    $N \leftarrow \|X\|$
    $\nu \leftarrow N - i - 1$
    $p \leftarrow 1 - \frac{\alpha}{2(\nu+2)}$
    $t_{p,\nu}$ denoting the critical value of the t distribution with ν degrees of freedom and a significance level of p
    $\tau \leftarrow \frac{(\nu+1)t_{p,\nu}}{\sqrt{(\nu+2)(\nu+t_{p,\nu}^2)}}$
    if $\frac{|x^* - \bar{X}|}{s} > \tau$ **then**
      $l \leftarrow \bar{X} - s \cdot \tau$ ▷ Lower bound
      $u \leftarrow \bar{X} + s \cdot \tau$ ▷ Upper bound
    end if
    Remove x^* from X
  end for
  $X \leftarrow XX$ ▷ Restore X
  for $i = 1$ **to** $|X|$ **do**
    if $X_i < l$ **or** $X_i > u$ **then**
      append i to O
    end if
  end for
  return O
end function

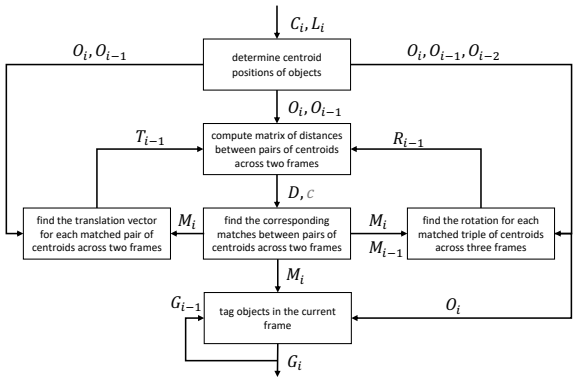
Algorithm 9 Labeling

function LABEL(C_i, M, O)
Input: Centroids of current frame C_i, match pairs M, and index of outliers O
Output: Labels L
  for $j = 1$ **to** $|C|$ **do**
    if $j \in O$ **then**
      $L_j \leftarrow$ *'Object'*
    else if $j \in \{j \,|\, (i, j) \in M\}$ **then**
      $L_j \leftarrow$ *'Star'*
    else
      $L_j \leftarrow$ *'Unknown'*
    end if
  end for
  return L
end function

Block Diagram Notations:

- $C_i \rightarrow$ Spot centroids for the ith frame.
- $L_i \rightarrow$ Label of the spots in the ith frame. Each label could be *'Object'*, *'Star'*, or *'Unknown'*.
- $O_i \rightarrow$ Object centroids for the ith frame.
- $D \rightarrow$ Distance matrix. $D_{p,q} \rightarrow$ Euclidean distance between the pth object centroid in the previous frame with q^{th} object centroid of current frame.
- $c \rightarrow$ Cost value selected for no matching found.
- $M_i \rightarrow$ Matches contains (p,q) pairs such that p^{th} object centroid in the previous frame is matched with the qth object centroid in the current frame.
- $T_i \rightarrow$ Translation vectors calculated for the ith frame. For each matched pair, there is a translation vector defined from the object centroid in the previous frame to the matched object centroid in the current frame.
- $R_i \rightarrow$ Rotations calculated for the ith frame. For each matched triple in three consecutive frames, there is a rotation defined from the object centroid in the previous frame to the matched object centroid in the current frame.
- $G_i \rightarrow$ Tag of the objects in the ith frame.

(a)



(b)

Fig. 3: Block diagram of tracking algorithm.

2.2 Tracking Algorithm

The purpose of the proposed tracking algorithm is to track the spots classified as objects in the classification step, across consecutive video frames. Algorithm 10 and Fig. 3 describe the computational methods and procedures used in the tracking algorithm.

Algorithm 10 Tracking

function TRACK(V)
  Input: Video V ▷ Sequence of intensity images
  Output: Tag of objects G
  constant c ▷ Cost of *Unknown* label (not matching)
  $L, C \leftarrow$ CLASSIFY(V)
  for $i = 1$ **to** $|V|$ **do** ▷ For each frame
    $O_i \leftarrow$ OBJECTS(L_i, C_i)
    $D \leftarrow$ DISTANCEMATRIX($O_{i-1}, O_i, T_{i-1}, R_{i-1}$)
    $M_i \leftarrow$ MATCHES(D, c)
    $T_i \leftarrow$ TRANSLATIONS(O_{i-1}, O_i, M_i)
    $R_i \leftarrow$ ROTATIONS($O_{i-2}, O_{i-1}, O_i, M_{i-1}, M_i$)
    $G_i \leftarrow$ TAGS(G_{i-1}, M_i)
  end for
  return G
end function

To track objects in each image frame, the first step is to determine the position of the objects in each frame. Algorithm 11 takes the centroids and labels of the spots obtained in the classification step, and returns only the centroids of the spots labeled as objects.

The distance matrix, matches, translation vectors, and rotations used in Algorithm 10 are calculated using the algorithms described respectively in Algorithm 3, 4, 5, and 6 in the classification section.

Algorithm 11 Select Objects

function OBJECTS(l, c)
Input: Labels l, centroids c
Output: Centroid of objects o
    $o \leftarrow \emptyset$
    for $i = 1$ **to** $|l|$ **do** ▷ For each debris
        if $l_i =$ 'Object' **then**
            append c_i to o
        end if
    end for
    return o
end function

In the final step, Algorithm 12 generates a consistent tag for each object in a video frame based on the calculated matching and the tags generated in the previous frame.

Algorithm 12 Assign Tags

function TAGS(g', M, o)
Input: Tags of previous frame g', matches M, centroid of objects o
Output: Tags of current frame g
    for $j = 1$ **to** $|o|$ **do** ▷ For each object in the current frame
        if $(i, j) \in M$ **then**
            $g_j \leftarrow g'_i$
        else
            $g_j \leftarrow$ a new unique tag
        end if
    end for
    return g
end function

The software implementing these algorithms is publicly available at [14].

3. RESULTS

Information about the sample video files is summarized in Table 1.

Table 1: Input sample data specifications

Sample data	# of Spots (mean ± std)	# of Objects (mean ± std)
No Rotation	118.3 ± 2.10	5.3 ± 2.1
Single Z Axis Rotation	114.8 ± 5.40	5.3 ± 2.0
Slow Z Axis Rotation	118.3 ± 2.10	5.3 ± 2.1
Faster Z Axis Rotation	109.0 ± 5.40	4.9 ± 1.7
Single X Axis Rotation	117.7 ± 3.50	5.4 ± 1.9
Slow X Axis Rotation	119.6 ± 1.50	5.3 ± 2.0
Faster X Axis Rotation	148.9 ± 39.5	1.1 ± 1.0
Full Rotation	114.0 ± 4.80	5.1 ± 2.3
Slow Full Rotation	117.8 ± 3.30	5.2 ± 2.1
Faster Full Rotation	133.5 ± 32.6	1.1 ± 0.6

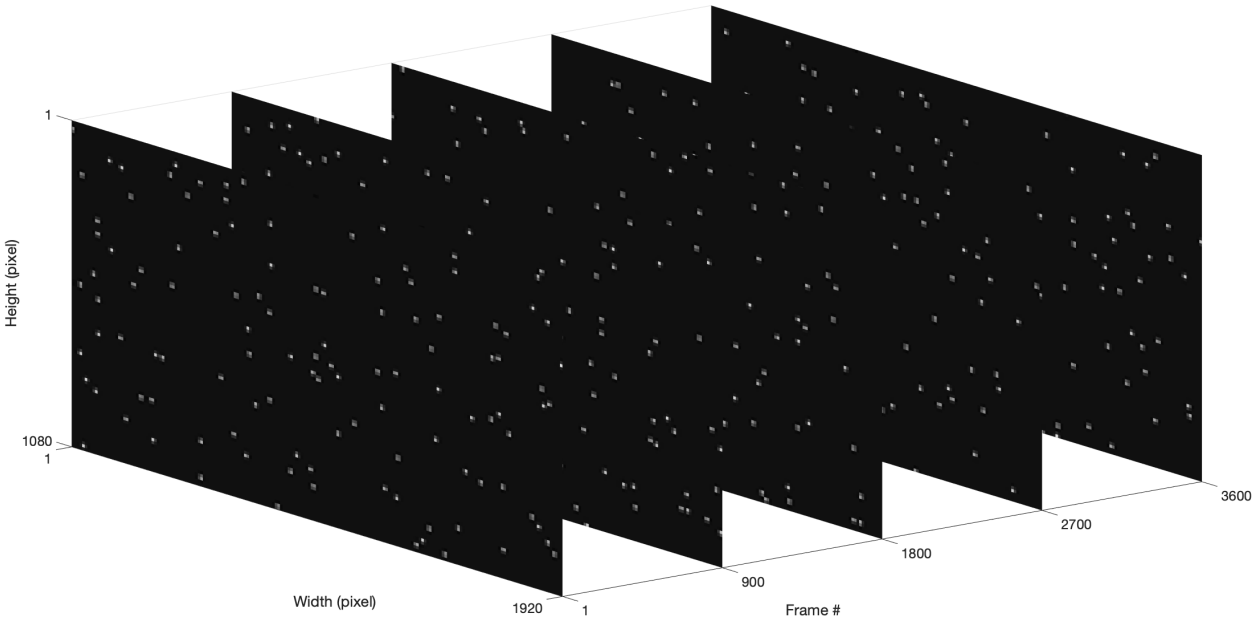
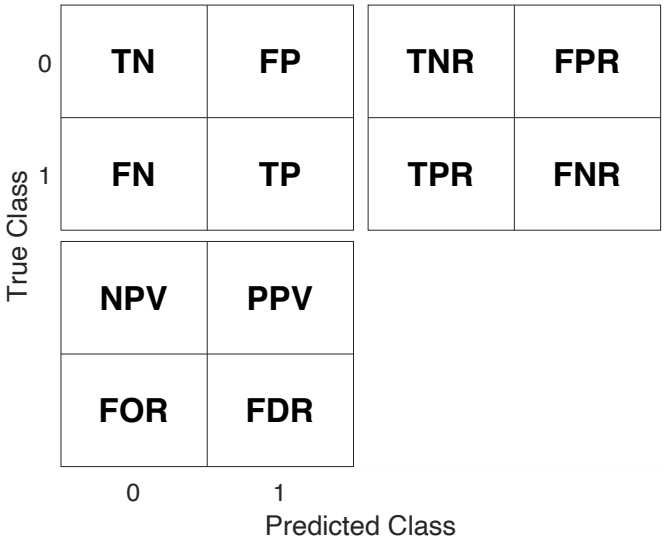


Fig. 4: Sample input data. Shown is a sequence of grayscale intensity image frames over time, represeting a few snapshots of the captured video sequence.

Confusion Matrix Notations:

- $TN \rightarrow$ True Negative.
- $FN \rightarrow$ False Negative.
- $FP \rightarrow$ False Positive.
- $TP \rightarrow$ True Positive.
- $TNR = \frac{TN}{TN+FP} \rightarrow$ True Negative Rate.
- $TPR = \frac{TP}{TP+FN} \rightarrow$ True Positive Rate (**sensitivity**).
- $FPR = 1 - TNR \rightarrow$ False Positive Rate.
- $FNR = 1 - TPR \rightarrow$ False Negative Rate.
- $NPV = \frac{TN}{TN+FN} \rightarrow$ Negative Predictive Value.
- $FOR = 1 - NPV \rightarrow$ False Omission Rate.
- $PPV = \frac{TP}{TP+FP} \rightarrow$ Positive Predictive Value (**precision**).
- $FDR = 1 - PPV \rightarrow$ False Discovery Rate.

(a)



(b)

Fig. 5: Description of the confusion matrix which is used to quantify the performance of the classification algorithm.

For all the sample videos *Frame size* is 1080 (pixel) Height $\times$ 1920 (pixel) Width, *Frame rate* is 60 frames per second (fps), *Duration* is 60 seconds, *Color format* is Grayscale, and the *Source data type* is floating-point number between 0 and 1. Sample snapshots of one of the sample video files (*Faster Z Axis Rotation* data in Table 1) is shown in Fig. 4.

Ten different datasets were used to evaluate the performance of the classification and tracking algorithms. Each of these datasets has different complexities in the characteristics of their foreground (number of objects and their motion pattern) and background (stars motion pattern). The title of each dataset in Table 1 indicates the status of the spacecraft when capturing the video frames.

The performance of the classification algorithm is measured using a confusion matrix whose rows and columns indicate, respectively, the actual labels for the spots from the ground truth data (True Class) and those predicted by the classification algorithm (Predicted Class) (Fig. 5).

Figure 6 summarizes the results of applying the classification algorithm on each of the datasets in Table 1, which report various performance measures described in figure 5.

Two measures that are particularly important for analyzing the performance of the classifier for the orbital debris classification application include *sensitivity*–the ratio of the number of spots correctly classified as an object to the total number of spots across all video frames, and *precision*–the ratio of the number of spots correctly classified as an object to the total number of spots classified as objects across all video frames.

The results show that the classification algorithm performed accurately in both *sensitivity* and *precision* senses on the sample datasets with more realistic motion patterns (denoted by 'slow rotation'), and failed to discriminate between objects and stars in the datasets denoted by 'faster rotation', which use unrealistic moving speeds compared to what is expected from the captured videos in real-world settings.

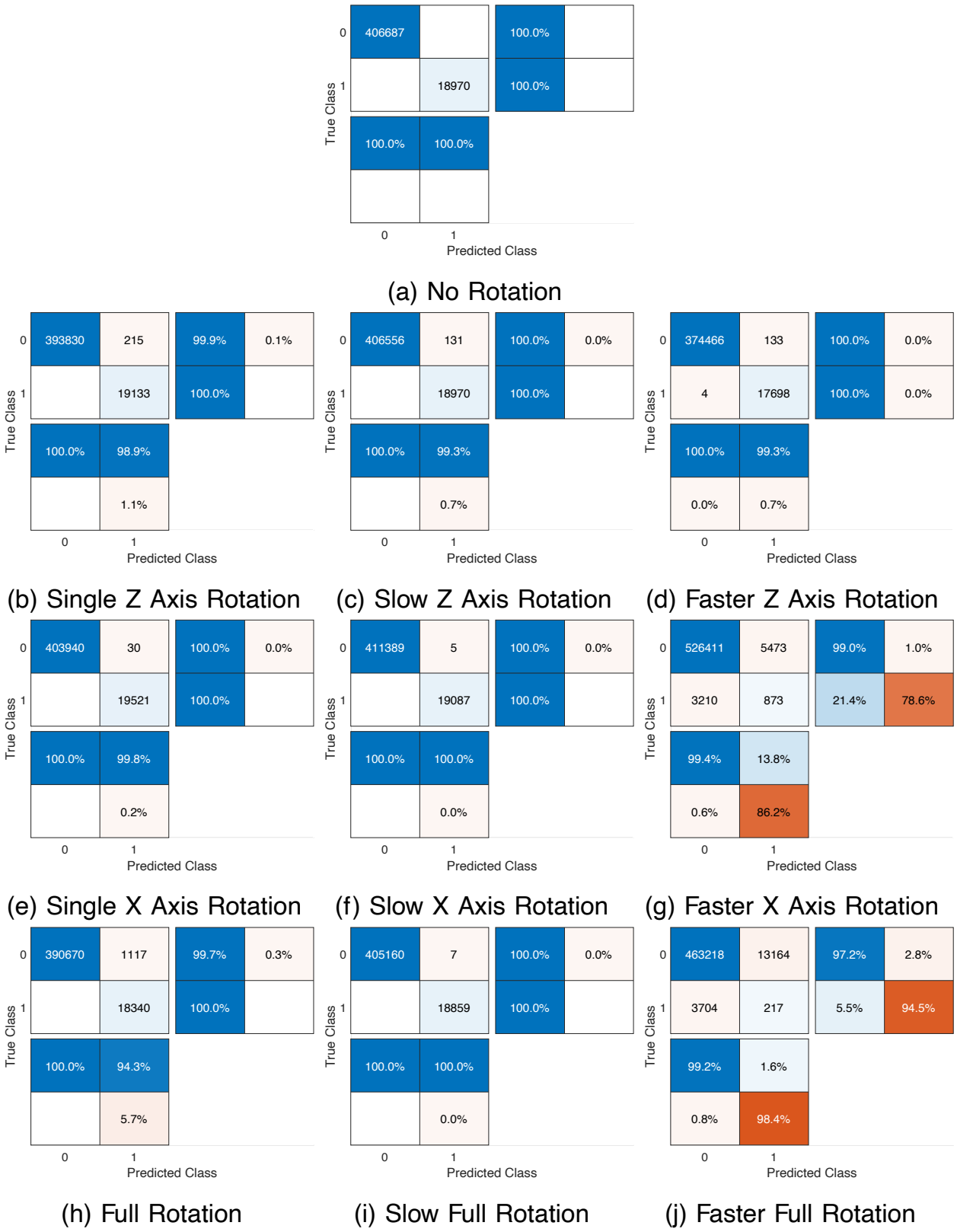


Fig. 6: Confusion matrices obtained by running the classification algorithm on a few sample input videos from Table 1. Here the spots classified as unknowns by the classification algorithm were manually included in the objects class in order to be able to define only two classes required for calculating the confusion matrix.

The *accuracy* of the tracking algorithm was defined as the ratio of the number of correctly tracked objects to the total number of correctly identified objects per frame. Table 2 reports the performance of the tracking algorithm, measured as the average *accuracy* across all the frames in the input video data for all the sample datasets described in Table 1. Similar to the classification performance results, except for the *'Faster Z Axis Rotation'* and *'Faster X Axis Rotation'* datasets, with unrealistically high speed simulated spacecraft motion, the tracking algorithm performed accurately across all the sample datasets.

The visualization of the output of the tracking algorithm is available at [15].

Table 2: Accuracy of tracking algorithm (per frame)

Sample data	**Accuracy% (mean $\pm$ std)**
No Rotation	100 ± 0
Single Z Axis Rotation	100 ± 0
Slow Z Axis Rotation	100 ± 0
Faster Z Axis Rotation	100 ± 0
Single X Axis Rotation	100 ± 0
Slow X Axis Rotation	100 ± 0
Faster X Axis Rotation	93 ± 24
Full Rotation	100 ± 0
Slow Full Rotation	100 ± 0
Faster Full Rotation	68 ± 46

For this analysis the values of unmatched cost (c), delay between two consecutive frames (d), significance level parameter (α), and the maximum possible number of outliers (K) used for detecting the outliers were set respectively to 1000, 1, 0.05 and 10% of the total number of spots in the current frame. The source code for generating these results is available at [14].

4. DISCUSSION

In this work, a statistical classifier and a tracking algorithm are developed to classify and track small bright spots in an input image sequence. When applied to the MSFC-provided simulated videos of small orbital debris objects moving over a static or dynamic star background, our algorithms showed successful in robustly classifying objects versus stars and consistently tracking the detected objects in consecutive frames.

The proposed algorithms have several degrees of freedom which can affect the classification and tracking performance. Two of those degrees of freedom (DoF), cost of unknown labels (c) and delay between two consecutive frames (d) were analyzed in the previous study [16]. For other DoF, values were chosen such that the algorithm showed the best performance on the sample data under study. However, these DoF can be tuned and optimized based on the statistics of the input images to be able to generalize to a variety of image settings or complexities. For example, for the threshold of binarization parameter (τ), a constant global value was selected here, but other dynamic thresholding methods could be used to adapt this value to any particular input image statistics [17]. In addition, the significance level parameter (α) and the maximum possible number of outliers (K) parameter used for detecting outliers are also among the DoF of the algorithm. Future studies can focus on optimizing possible DoF values based on certain applications and the desired performance measures.

Moreover, the features used in Algorithm 7 were extracted based on the motion model of stars and objects that best described the sample datasets under study. Future studies can modify these features or use other possible features based on the particular characteristics and the motion models best describe their data. There are also a variety of algorithms for video tracking–the process of locating one or multiple moving object over time– such as mean-shift tracking [18], Kalman filter [19], and Particle filter [20–22]. These algorithms analyze sequential video frames and output the movement trajectory of objects of interest between the frames. The follow-on studies can take advantage of these other tracking algorithms depending on particular characteristics of the data at hand.

5. REPORT FROM ADVANCED MAUI OPTICAL AND SPACE SURVEILLANCE TECHNOLOGY CONFERENCE (AMOS 2019)

Among the visitors two teams showed great interest in this work, discussing the outcomes and ideas around vision-based solutions for orbital debris problems. One was the EOS Space Systems (EOS SS) which aim to expand the network of space sensors to provide accurate, specialised catalogues for assessing collision risk in real time. Conjunction analysis is performed continuously, and the sensor network is re-tasked in real time to allow timely collision prediction using orbit errors smaller than 100m. EOS space tracking and debris monitoring systems provide the benchmark for space catalogue acquisition and maintenance, in all key performance indicators:

- **Accuracy**. Space object position can be determined with an absolute accuracy selectable from 1 mm to 1 m, depending on the application. No other operational space technology offers this accuracy.
- **Sensitivity**. EOS trackers have been validated for tracking objects as small as 6mm in size at 350km range, 5cm at 1,000km and 30 cm at 35,000 km. This sensitivity addresses over 90% of all collision risk to operational spacecraft.
- **Range**. EOS sensors are able to readily track all orbit heights and regimes from low Earth orbit (LEO) through to geosynchronous equatorial orbit (GEO) and beyond.
- **Cost**. Low cost per space track allows frequent tracking of each object and highly accurate catalogues.

The other visiting group was from Space Environment Research Centre (SERC). The CRC for Space Environment Management (SEMCRC), managed by the SERC, aims to build on Australian and international expertise in measurement, monitoring, analysis and management of space debris and to develop technologies to preserve the space environment. SERC is also addressing the challenges in managing space debris and preserving the space environment. They can monitor pieces of orbiting debris which are larger than 5mm.

The findings from this project could be of potential interest of these teams or open up possible future collaboration opportunities with them.

6. SUMMARY OF TECHNICAL OUTCOMES AND SCHEDULE

Table 3 summarizes the project schedule. Milestones are denoted by **X** in the table.

Table 3: Technical Schedule

	Year 1			
	Q1	Q2	Q3	Q4
Kick-off Meeting	**X**			
Objective 1 – Algorithm Design and Optimization				
1.1 - Develop computational model in Matlab	**X**			
1.2 - Simulate and validate model on synthetic images				
1.3 - Redefine modules for best performance on synthetic data				
Objective 2 – Testing algorithm on space image datasets				
2.1 - Create local database of debris images		**X**		
2.2 - Evaluate and analyze model on simulated debris images			**X**	
2.3 - Optimize and redefine module for best performance on simulated data				**X**
Closeout Meeting				X

Monthly Skype conferences were held with NASA personnel in order to keep our progress in line with the objectives of MSFC. Table 4 lists the specific outcomes for each project's sub-objective, the deliverables for each milestone specified in table 3, and the evaluation criteria that were used to measure the project progress.

Table 4: Final results and evaluation metrics for each project objective and milestone

	Final Outcome and Deliverables	**Evaluation Criteria**
Objective 1 – Algorithm Design and Optimization		
1.1 - Develop computational model in Matlab	A Matlab program was developed to implement the classification and tracking algorithm (**Milestone 1**).	Operational model of algorithm using synthetic data by end of Q1.
1.2 - Simulate and validate model on synthetic images	The functionality of the computational model was verified using synthetic images of moving objects	Achieving functional model of algorithm on synthetic images by end of Q2.
1.3 - Redefine modules for best performance on synthetic data	The performance of the model was optimized on synthetic data	Optimizing the model for various performance variables by end of Q2.
Objective 2 – Testing algorithm on space images from MSFC DSFS		
2.1 - Create database of debris images generated by MSFC DSFC	Sample simulated image data for orbital debris environments were provided by MSFC (**Milestone 2**).	Creating a set of ground truth Matlab data files to be used for testing the developed algorithms by end of Q2.
2.2 - Evaluate and analyze model on simulated debris images	The functionality of the classification algorithm was verified using simulated images of moving orbital objects (**Milestone 3**).	Achieving the target performance of the classification algorithm by end of Q3. Results were presented at AMOS 2019.
2.3 - Optimize and redefine module for best performance on simulated data	The functionality of the tracking algorithm was verified using simulated images of moving orbital objects. The performance of the combined classification and tracking algorithms was optimized across all the datasets (**Milestone 4**).	Achieving the target performance of tracking orbital debris objects across all the simulated image datasets under various simulated complexity variables by end of Q4.

ACKNOWLEDGMENT

The work was supported by NASA 80MSFC18M0010 grant to N.N.

REFERENCES

[1] A. Nafi, A. Bernard, and K. Fujimoto, "A unified approach for optical survey strategy design of resident space objects," in *AIAA/AAS Astrodynamics Specialist Conference*, p. 5500, 2016.

[2] A. C. Snow, J. L. Worthy III, A. den Boer, L. J. Alexander, M. J. Holzinger, and D. Spencer, "Optimization of cubesat constellations for uncued electrooptical space object detection and tracking," *Journal of Spacecraft and Rockets*, pp. 401–419, 2016.

[3] A. D. Biria and B. G. Marchand, "Constellation design for space-based space situational awareness applications: an analytical approach," *Journal of Spacecraft and Rockets*, vol. 51, no. 2, pp. 545–562, 2014.

[4] C. Solomon and T. Breckon, *Fundamentals of Digital Image Processing: A practical approach with examples in Matlab*. John Wiley & Sons, 2011.

[5] A. Rosenfeld, J. L. Pfaltz, *et al.*, "Sequential operations in digital picture processing.," *J. ACM*, vol. 13, no. 4, pp. 471–494, 1966.

[6] J. E. Gentle, "Matrix algebra," *Springer texts in statistics, Springer, New York, NY, doi*, vol. 10, pp. 978–0, 2007.

[7] J. Matousek and B. Gärtner, *Understanding and using linear programming*. Springer Science & Business Media, 2007.

[8] R. Jonker and A. Volgenant, "A shortest augmenting path algorithm for dense and sparse linear assignment problems," *Computing*, vol. 38, no. 4, pp. 325–340, 1987.

[9] E. T. Whittaker, *A treatise on the analytical dynamics of particles and rigid bodies*. Cambridge University Press, 1988.

[10] C. Kimberling, *Triangle centers and central triangles*. Utilitas Mathematica Publishing Incorporated, 1998.

[11] G. L. Tietjen and R. H. Moore, "Some grubbs-type statistics for the detection of several outliers," *Technometrics*,

vol. 14, no. 3, pp. 583–597, 1972.
[12] W. Stefansky, "Rejecting outliers in factorial designs," *Technometrics*, vol. 14, no. 2, pp. 469–479, 1972.
[13] B. Rosner, "Percentage points for a generalized esd many-outlier procedure," *Technometrics*, vol. 25, no. 2, pp. 165–172, 1983.
[14] Y. Zamani and N. Nategh, "Amos2020." `https://github.com/nnategh/AMOS2020`, 2020.
[15] Y. Zamani, "Amos2020." `https://www.youtube.com/watch?v=VFNVB4zX9Qk&list=PLR0Ezs-OTZC41fKdtn9gpoWx8A2s4e0HG`, 2020.
[16] Y. Zamani, J. Amert, T. Bryan, and N. Nategh, "A robust vision-based algorithm for detecting and classifying small orbital debris using on-board optical cameras," *Advanced Maui Optical and Space Surveillance Technologies Conference (AMOS)*, 2019.
[17] D. Bradley and G. Roth, "Adaptive thresholding using the integral image," *Journal of graphics tools*, vol. 12, no. 2, pp. 13–21, 2007.
[18] D. Comaniciu, V. Ramesh, and P. Meer, "Real-time tracking of non-rigid objects using mean shift," in *Proceedings IEEE Conference on Computer Vision and Pattern Recognition. CVPR 2000 (Cat. No. PR00662)*, vol. 2, pp. 142–149, IEEE, 2000.
[19] M. S. Arulampalam, S. Maskell, N. Gordon, and T. Clapp, "A tutorial on particle filters for online nonlinear/non-gaussian bayesian tracking," *IEEE Transactions on signal processing*, vol. 50, no. 2, pp. 174–188, 2002.
[20] E. Maggio and A. Cavallaro, *Video tracking: theory and practice*. John Wiley & Sons, 2011.
[21] H. Kim, R. Sakamoto, I. Kitahara, T. Toriyama, and K. Kogure, "Robust foreground segmentation from color video sequences using background subtraction with multiple thresholds," *Proc. KJPR*, pp. 188–193, 2006.
[22] J. M. del Rincón, D. Makris, C. O. Uruñuela, and J.-C. Nebel, "Tracking human position and lower body parts using kalman and particle filters constrained by human biomechanics," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 41, no. 1, pp. 26–37, 2010.
[23] E. Alpaydin, *Introduction to machine learning*. MIT press, 2009.